HossainMohammad

Shahjalal University of Science and Technology

1 Number Theory **1**

1.1 Binomial Coeffecient 1
1.2 Phi sieve, Phi brute 1
1.3 Mobius Inversion 1
1.4 Segmented Sieve 1
1.5 Extended GCD 1
1.6 CRT 1
1.7 Diophantine Equation 1
1.8 Miller Rabin Pollar Rho . . 2
1.9 Mod Square Root 2
1.10 Discrete Logarithm 2
1.11 Floor Sum 2
1.12 Ballot Number 2
1.13 Multiplicative Order 2
1.14 Power Tower 2
1.15 Dirichlet Convolution 2

1 Number Theory

1.1 Phi sieve, Phi brute

```
int phi[MAXN];
void sieve() {
  for (int i = 0; i < MAXN; ++i) phi[i]
     = i;
  for (int i = 2; i < MAXN; ++i)
    if (phi[i] == i)
      for (int j = i; j < MAXN; j += i)
     phi[j] -= phi[j] / i;
}
int phi(int n) {
  int res = n;
  for (int i = 2; i * i <= n; i++) {
    if (n % i == 0) {
      while (n % i == 0) n /= i;
      res -= res / i;
    }
  }
  if (n > 1) res -= res / n;
  return res;
}
```

1.2 Mobius Inversion

```
int mob[N];
// Precomputes Mobius function (i) for
     all 1 <= i < N.
void mobius() {
  mob[1] = 1;
  for (int i = 2; i < N; i++){
    mob[i]--;
    for (int j = i + i; j < N; j += i)
     mob[j] -= mob[i];
  }
}
bool vis[N];
vector<int> d[N];
int mul[N];
// Inserts (k=1) or removes (k=-1) x by
     updating divisor frequencies.
void add(int x, int k) {
  for (auto y: d[x]) mul[y] += k;
}
// Returns the count of active set
     elements coprime to x.
int query(int x) {
  int ans = 0;
  for (auto y: d[x]) ans += mul[y] * mob
     [y];
  return ans;
}
```

1.3 Segmented Sieve

```
// Generate all primes from l to r using
     segmented sieve in
// O((r - l) log (r) + sqrt(r)) // needs
     sieve(limit)
vector<int64_t> segmented_sieve(int64_t
    l, int64_t r) {
  if (l == 1) l++;
  int limit = sqrtl(r);
  while (limit * limit <= r) limit++;
  while (limit * limit > r) limit--;
  auto primes = sieve(limit);
  vector<bool> is_prime(r - l + 1, true)
     ;
  for (int64_t p : primes) {
    int64_t start = max(p * p, (l + p -
     1) / p * p);
    for (int64_t j = start; j <= r; j +=
     p) {
      is_prime[j - l] = false;
    }
  }
  vector<int64_t> vec;
  for (int64_t i = l; i <= r; ++i) {
    if (is_prime[i - l]) vec.push_back(i
     );
  }
  return vec;
}
```

1.4 Extended GCD

Description: Finds solutions (x, y) in $ax + by = gcd(a, b)$. (Check formulae for more details)

```
ll extgcd(ll a, ll b, ll &x, ll &y) {
  if (!b) { x = 1; y = 0; return a; }
  ll x1, y1, g = extgcd(b, a % b, x1, y1
     );
  x = y1; y = x1 - (a / b) * y1;
  return g;
}
```

1.5 CRT

Description: Computes the unique solution x to a system of congruences $x \equiv a_i \pmod{m_i}$ using the Chinese Remainder Theorem. Assumes all moduli are pairwise coprime and the answer fits in ll.
Usage: Store congruences as {remainder, modulus} pairs in mods. Call crt(mods) to obtain the smallest non-negative solution modulo the product of all moduli. Requires extended_gcd() and modinv().
Time: $O(k \log M)$

```
bool merge(ll a1, ll m1, ll a2, ll m2,
     ll &res, ll &mod) {
  ll x, y, g = extgcd(m1, m2, x, y);
  if ((a2 - a1) % g != 0) return false;
  ll m2g = m2 / g;
  ll t = (ll)((__int128)(a2 - a1) / g %
     m2g) * (x % m2g) % m2g);
  if (t < 0) t += m2g;
  __int128 lcm_mod = (__int128)(m1 / g)
     * m2;
  __int128 tmp = (__int128)a1 + (
     __int128)m1 * t;
  res = (ll)(tmp % lcm_mod);
  if (res < 0) res += (ll)lcm_mod;
  mod = (ll)lcm_mod;
  return true;
}
```

1.6 Diophantine Equation

Description: Solves $ax+by = c$ where $gcd(a, b)|c$. Find any solution returns $x \in [minx, maxx]$ and $y \in [miny, maxy]$. If $solves = true$, the function returns all solution pairs (x, y). (Check formulae for more details)

```
// Solves a*x + b*y = c
bool find_any_solution(ll a,ll b,ll c,ll
      &x0,ll &y0,ll &g){
    if(a==0&&b==0){
        if(c) return false;
        x0=y0=g=0; return true;
    }
    g=extended_gcd(abs(a),abs(b),x0,y0);
    if(c%g!=0) return false;
    x0*=c/g; y0*=c/g;
    if(a<0) x0*=-1;
    if(b<0) y0*=-1;
    return true;
}
// Shift solution by cnt
void shift_solution(ll &x,ll &y,ll a,ll
     b,ll cnt){
    x+=cnt*b; y-=cnt*a;
}
pair<ll, vector<pair<ll, ll>>>
     find_all_solutions(ll a,ll b,ll c,
                        ll minx,ll maxx,ll
      miny,ll maxy, bool solves){
    ll x,y,g;
    if(!find_any_solution(a,b,c,x,y,g))
     return {0, {}};
    if(a==0&&b==0){
        assert(c==0);
        ll cnt = 1LL*(maxx-minx+1)*(maxy
     -miny+1);
        vector<pair<ll, ll>> res;
        if(solves){
            for(ll cur_x=minx; cur_x<=
     maxx; ++cur_x){
                for(ll cur_y=miny; cur_y
     <=maxy; ++cur_y){
                    res.push_back({cur_x
     , cur_y});
                }
            }
        }
        return {cnt, res};
    }
    if(a==0){
        ll cnt = (maxx-minx+1)*(miny<=c/
     b&&c/b<=maxy);
        vector<pair<ll, ll>> res;
        if(solves && cnt > 0){
            ll cur_y = c/b;
            for(ll cur_x=minx; cur_x<=
     maxx; ++cur_x){
                res.push_back({cur_x,
     cur_y});
            }
        }
        return {cnt, res};
    }
    if(b==0){
        ll cnt = (maxy-miny+1)*(minx<=c/
     a&&c/a<=maxx);
        vector<pair<ll, ll>> res;
        if(solves && cnt > 0){
            ll cur_x = c/a;
            for(ll cur_y=miny; cur_y<=
     maxy; ++cur_y){
                res.push_back({cur_x,
     cur_y});
            }
        }
        return {cnt, res};
    }
    ll orig_a = a, orig_b = b;
    a/=g; b/=g;
    ll sign_a=a>0?1:-1, sign_b=b>0?1:-1;
    shift_solution(x,y,a,b,(minx-x)/b);
    if(x<minx) shift_solution(x,y,a,b,
     sign_b);
    if(x>maxx) return {0, {}};
    ll lx1=x;
    shift_solution(x,y,a,b,(maxx-x)/b);
    if(x>maxx) shift_solution(x,y,a,b,-
     sign_b);
    ll rx1=x;
    shift_solution(x,y,a,b,-(miny-y)/a);
    if(y<miny) shift_solution(x,y,a,b,-
     sign_a);
    if(y>maxy) return {0, {}};
    ll lx2=x;
    shift_solution(x,y,a,b,-(maxy-y)/a);
    if(y>maxy) shift_solution(x,y,a,b,
     sign_a);
    ll rx2=x;
    if(lx2>rx2) swap(lx2,rx2);
    ll lx=max(lx1,lx2), rx=min(rx1,rx2);
    if(lx>rx) return {0, {}};
    ll cnt = (rx-lx)/abs(b)+1;
```

```
    vector<pair<ll, ll>> res;
    if(solves){
        for(ll cur_x=lx; cur_x<=rx;
     cur_x+=abs(b)){
            res.push_back({cur_x, (c-
     orig_a*cur_x)/orig_b});
        }
    }
    return {cnt, res};
}
```

1.7 Miller Rabin Pollar Rho

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -¿ {11, 19, 11}). Time: $O(n^{1/4})$, less for numbers with small factors.

```
bool isPrime(ull n) {
  if (n < 2 || n % 6 % 4 != 1) return (n
      | 1) == 3;
  ull A[] = {2, 325, 9375, 28178,
     450775, 9780504, 1795265022},
      s = __builtin_ctzll(n - 1), d = n
     >> s;
  for (ull a : A) {
    ull p = modpow(a % n, d, n), i = s;
    while (p != 1 && p != n - 1 && a % n
      && --i) p = mul(p, p, n);
    if (p != n - 1 && i != s) return 0;
  }
  return 1;
}
ull pollard(ull n) {
  ull x = 0, y = 0, t = 30, prd = 2, i =
      1, q;
  auto f = [&](ull x) {
    ull res = mul(x, x, n);
    return res + i >= n ? res + i - n :
     res + i;
  };
  while (t++ % 40 || __gcd(prd, n) == 1)
      {
    if (x == y) x = ++i, y = f(x);
    if ((q = mul(prd, max(x, y) - min(x,
      y), n))) prd = q;
    x = f(x), y = f(f(y));
  }
  return __gcd(prd, n);
}
vector<ull> factor(ull n) {
  if (n == 1) return {};
  if (isPrime(n)) return {n};
  ull x = pollard(n);
  auto l = factor(x), r = factor(n / x);
  l.insert(l.end(), all(r));
  return l;
}
```

1.8 Mod Square Root

Description: Tonelli-Shanks algorithm for modular square roots. Finds x s.t. $x^2 = a \pmod p$ ($-x$ gives the other solution). **Time:** $O(\log^2 p) worst case, O(\log p) for most p$

```
ll sqrt(ll a, ll p) {
  a %= p; if (a < 0) a += p;
  if (a == 0) return 0;
  assert(modpow(a, (p-1)/2, p) == 1); //
      else no solution
  if (p % 4 == 3) return modpow(a, (p+1)
     /4, p);
  // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4
     works if p % 8 == 5
  ll s = p - 1, n = 2;
  int r = 0, m;
  while (s % 2 == 0)
    ++r, s /= 2;
  /// find a non-square mod p
  while (modpow(n, (p - 1) / 2, p) != p
     - 1) ++n;
  ll x = modpow(a, (s + 1) / 2, p);
  ll b = modpow(a, s, p), g = modpow(n,
     s, p);
  for (;; r = m) {
    ll t = b;
    for (m = 0; m < r && t != 1; ++m)
      t = t * t % p;
    if (m == 0) return x;
    ll gs = modpow(g, 1LL << (r - m - 1)
     , p);
    g = gs * gs % p;
    x = x * gs % p;
    b = b * g % p;
  }
}
```

1.9 Discrete Logarithm

Description: Solves $a^x \equiv b \pmod m$ for the smallest $x > 0$, using the Baby-Step Giant-Step algorithm. Supports non-coprime a and m by explicitly checking small exponents $x \leq \sqrt{m}$ before applying step shifts. Can also compute the multiplicative order of a via `modLog(a, 1, m)`.

Usage:
`modLog(a, b, m) // returns x > 0 or -1`

```
ll modLog(ll a, ll b, ll m) {
  ll n = (ll) sqrt(m) + 1, e = 1, f = 1,
      j = 1;
  unordered_map<ll, ll> A;
  while (j <= n && (e = f = e * a % m)
     != b % m)
    A[e * b % m] = j++;
  if (e == b % m) return j;
  if (__gcd(m, e) == __gcd(m, b))
    rep(i,2,n+2) if (A.count(e = e * f %
      m))
      return n * i - A[e];
  return -1;
}
```

1.10 Floor Sum

Description: Calculates $\sum_{i=0}^{n-1} \left\lfloor \frac{a \cdot i + b}{m} \right\rfloor$ in $O(\log m)$ time Handles negative a, b and values $\geq m$ by automatically normalizing parameters modulo m. **Usage:** `fsum(n, m, a, b)`. **Time:** $O(\log m)$

```
ll fsum(ll n, ll m, ll a, ll b) {
  ll r = 0;
  if (a < 0) { ll a2 = (a % m + m) % m;
     r -= n * (n - 1) / 2 * ((a2 - a) /
     m); a = a2; }
  if (b < 0) { ll b2 = (b % m + m) % m;
     r -= n * ((b2 - b) / m); b = b2; }
  if (a >= m) { r += (n - 1) * n / 2 * (
     a / m); a %= m; }
  if (b >= m) { r += n * (b / m); b %= m
     ; }
  ll y = (a * n + b) / m;
  if (!y) return r;
  ll x = y * m - b;
  return r + (n - (x + a - 1) / a) * y +
      fsum(y, a, m, (a - x % a) % a);
}
```

1.11 Ballot Number

```
// Returns the number of paths from
     (0,0) to (a,b) staying at or below y
      = x + k
long long ballot_with_offset(int a, int
     b, int k) {
  if (b > a + k) return 0; // Already
     invalid
  long long total = nCr(a + b, a);
  long long invalid = nCr(a + b, a + k +
      1);
  return (total - invalid + mod) % mod;
}
// Strict Ballot Number: A > B at all
     non-empty prefixes
long long ballot_strict(int a, int b) {
  if (a <= b) return 0;
  return ballot_with_offset(a - 1, b,0)
     ;
}
```

1.12 Multiplicative Order

```
ll multiplicative_order(ll a, ll mod) {
  if (__gcd(a, mod) != 1) return -1;
  ll m = totient(mod), p = m;
  ll ans = 2e18;
  if (power(a, p, mod) == 1) ans = p;
  vector<ll> fac;
  for (ll i = 2 ; i * i <= m; i++) {
    if (m % i == 0) {
      while(m % i == 0) m /= i, fac.
     push_back(i);
    }
  }
  if (m > 1) fac.push_back(m);
  for (auto x: fac) {
    if (power(a, p / x, mod) == 1) p /=
     x, ans = p;
  }
  assert(ans != 2e18);
  return ans;
}
```

1.13 Power Tower

```
map<ll, ll> mp;
ll phi(ll n) {
  if (mp.count(n)) return mp[n];
  ll ans = n, m = n;
  for (ll i = 2; i * i <= m; i++) {
    if (m % i == 0) {
      while (m % i == 0) m /= i;
      ans = ans / i * (i - 1);
    }
  }
  if (m > 1) ans = ans / m * (m - 1);
  return mp[n] = ans;
}
inline ll MOD(ll x, ll m) {
  if (x < m) return x;
  return x % m + m;
}
int a[N];
// if x >= log2(m), then a^x = a^(MOD(x,
     phi(m))) % m
// Usage: ll ans = powerTower(l, r, m) %
     m; // computes (a[l] ^ a[l+1] ^ ...
     ^ a[r]) % m
ll powerTower(ll l, ll r, ll m) {
  if (l == r) return MOD(a[l], m);
  if (m == 1) return 1;
  return power(a[l], powerTower(l + 1, r
     , phi(m)), m);
}
```

1.14 Dirichlet Convolution

```
/**
Prefix sum of multiplicative functions :
  p_f : the prefix sum of f(x) (1 <= x
     <= T).
  p_g : the prefix sum of g(x) (0 <= x
     <= N).
  p_c : the prefix sum of f * g(x) (0 <=
      x <= N).
  T : the threshold, generally should be
      n ^ (2 / 3). for n = 1e9, T = 1e6
Magic Functions for different f(x)
For f(x) = phi(x): Id(x) = phi * I(x) i.
     e. p_c = prefix sum of Id(x), p_g =
     prefix sum of I(x). Here Id(n) = n,
     I(n) = 1
For f(x) = mu(x): e(x) = mu * I(x). Here
      e(x) = x == 1 ? 1 : 0
Complexity: O(n^(2/3))
**/
namespace Dirichlet {
  //solution for f(x) = phi(x)
  const int T = 1e7 + 9;
  long long phi[T];
  gp_hash_table<long long, mint> mp;
  mint dp[T], inv;
  int sz, spf[T], prime[T];
  void init() {
    memset(spf, 0, sizeof spf);
    phi[1] = 1; sz = 0;
    for (int i = 2; i < T; i++) {
      if (spf[i] == 0) phi[i] = i - 1,
     spf[i] = i, prime[sz++] = i;
      for (int j = 0; j < sz && i *
     prime[j] < T && prime[j] <= spf[i];
      j++) {
        spf[i * prime[j]] = prime[j];
        if (i % prime[j] == 0) phi[i *
     prime[j]] = phi[i] * prime[j];
        else phi[i * prime[j]] = phi[i]
     * (prime[j] - 1);
      }
```

```
    }
    dp[0] = 0;
    for(int i = 1; i < T; i++) dp[i] =
     dp[i - 1] + phi[i] % mod;
    inv = 1; // g(1)
  }
  mint p_c(long long n) {
    if (n % 2 == 0) return n / 2 % mod *
      ((n + 1) % mod) % mod;
    return (n + 1) / 2 % mod * (n % mod)
      % mod;
  }
  mint p_g(long long n) {
    return n % mod;
  }
  mint solve (long long x) {
    if (x < T) return dp[x];
    if (mp.find(x) != mp.end()) return
     mp[x];
    mint ans = 0;
    for (long long i = 2, last; i <= x;
     i = last + 1) {
      last = x / (x / i);
      ans += solve (x / i) * (p_g(last)
     - p_g(i - 1));
    }
    ans = p_c(x) - ans;
    ans /= inv;
    return mp[x] = ans;
  }
}
```